SCALE FOR PROJECT PYTHON MODULE 02 (/PROJECTS/PYTHON-MODULE-02)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere.42heilbronn.de:vogsphere/intra-uuid-58775fed-4cd9-4d85
```

Introduction

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.

- Identify, together with the person (or group) being evaluated, any issues in the work. Take the time to discuss and debate the problems you have identified.

- You must consider that there might be differences in how your peers have understood the project's instructions and the scope of its functionality. Always keep an open mind and grade them as honestly as possible. The pedagogy is valid only if peer evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student's or group's Git repository.

- Double-check that the Git repository belongs to the student or the group. Ensure that the work is for the relevant project and also check that "git clone" is run in an empty folder.

- Check carefully that no malicious aliases were used to fool you and make you evaluate something other than the content of the official repository.

- To avoid any surprises, carefully check that both the evaluating and the evaluated students have reviewed the possible scripts used to facilitate the grading.

- If the evaluating student has not completed that particular project yet, it is mandatory for this student to read the entire subject document prior to starting the defence.

- Use the flags available on this scale to signal an empty repository, a non-functioning program, a norm error, cheating, etc. In these cases, the grading is over and the final grade is 0 (or -42 in case of cheating). However, with the exception of cheating, you are

encouraged to continue to discuss your work (even if you have not
finished it) in order to identify any issues that may have caused
this failure and avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no unexpected,
premature, uncontrolled termination of the program is allowed;
otherwise, the final grade is 0. Use the appropriate flag.
You should never have to edit any file except the configuration file if it exists.
If you want to edit a file, take the time to explain the reasons with the
evaluated student and make sure both of you agree with this change.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/212654/en.subject.pdf)

Preliminaries

Basics

- Only grade the work that is in the learner's Git repository
- Ensure the Git repository belongs to the learner
- Check that no malicious aliases are used
- Verify the project structure matches the subject requirements

Are all preliminary checks satisfied?

Yes No

General Instructions

During the evaluation of each exercise, ensure the following
general instructions are met:
• Programs must be written in Python 3.10 or higher
• The code must comply with the flake8 linter standards (with no warnings or errors)
• Each exercise should be in its own file as specified
• Type hints are REQUIRED for all functions and methods (parameters and return values). Check this using mypy.
• Docstrings are NOT required for this module
• Programs must never crash unexpectedly (no uncaught exceptions)

Yes No

Exercises

Exercise 0 - Agricultural Data Validation

Check ft_first_exception.py:

• Function input_temperature(temp_str) exists and works correctly
• Function test_temperature handles Exception or ValueError in a try/except block
• Code tests input_temperature() with valid and invalid inputs
• test_temperature() prints an error message when an exception occurs and continues running
• input_temperature() returns the temperature when input is correct

Understanding check:
• Can the learner explain what happens when you try to convert "abc" to a number?
• Can they explain why the program does not crash when errors occur?

☑ Yes ☒ No

Exercise 1 - Agricultural Data Validation Pipeline

Check ft_raise_exception.py:

• Function input_temperature(temp_str) is updated from Exercise 0 and now checks min and max temperature values
• Function input_temperature(temp_str) raises an exception if temperature is too hot or too cold
• Function test_temperature handles Exception or ValueError in a try/except block
• Code tests input_temperature() with valid and invalid inputs, including values that are too hot and too cold
• test_temperature() prints an error message when an exception occurs and continues running
• input_temperature() still returns the temperature when the input is valid (a numeric value within the limits)

☑ Yes ☒ No

Exercise 2 - Different Types of Problems

Check ft_different_errors.py:

Error type demonstration:
• The garden_operations() function creates different error scenarios
• Demonstrates ValueError (bad data conversion)
• Demonstrates ZeroDivisionError (division by zero)
• Demonstrates FileNotFoundError (missing file)
• Demonstrates TypeError (type mismatch) - Reminder: the mypy error on this faulty code is allowed

Error handling:
• test_error_types() function exists
• A single try: block exists, with multiple except: clauses, one for each error type
• The function correctly catches each error scenario raised by garden_operation()
• The program continues running after each error

Understanding check:
• Can the learner explain why Python has different error types?

☑ Yes ☒ No

Exercise 3 - Making Your Own Error Types

Check ft_custom_errors.py:

Custom exception classes:
• GardenError class exists and inherits from Exception
• PlantError class exists and inherits from GardenError
• WaterError class exists and inherits from GardenError
• Classes are simple and well-structured

Usage demonstration:
• A testing structure exists: exceptions of different types are raised and then caught
(a structure identical to that in the previous exercise is recommended but not mandatory)
• Tests show catching specific custom error types (PlantError, WaterError)
• Tests demonstrate inheritance (catching GardenError catches all garden errors)

Recode by the evaluated learner:
• The subject specifies a default message when none is provided. Without modifying the three custom classes, add a test that raises a PlantError with no parameter.
Is the default message properly displayed?

☑ Yes ☒ No

Exercise 4 - Finally Block - Always Clean Up

Check ft_finally_block.py:

• water_plant(plant_name) function exists
• It checks for a capitalized plant name using plant_name.capitalize() == plant_name
• It raises a PlantError exception if not
• It prints a success message otherwise

• test_watering_system() function exists
• An "opening" message is printed before watering plants
• A try/except/finally structure exists and handles errors properly
• On error, test_watering_system() stops and returns to the main part of the file
• The finally: block is always executed (it prints a "closing" message), even when the function returns on error
• Tests include normal operation (no errors) and an error scenario (a non-capitalized plant name)

Understanding check:
• Can the learner explain why cleanup is important even when errors happen?
• Do they understand when the finally block executes?
• Can they give examples of resources that need cleanup?

Yes No

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat Crash

Concerning situation Leaks Forbidden function Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Atousa did a good job! She explained her code well

Finish evaluation